

Pandas Full Notes (Beginner to Advanced) - Detailed Version

1. Introduction to Pandas

Pandas is a powerful open-source Python library used for data analysis and data manipulation. It provides flexible data structures like Series and DataFrame which make handling structured data easy. It is widely used in Data Science, Machine Learning, Finance, and Web Analytics.

2. Series (1D Data Structure)

A Series is a one-dimensional labeled array capable of holding any data type. Each element in a Series has an associated index.

Example:

```
import pandas as pd
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
```

3. DataFrame (2D Data Structure)

A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. It is similar to a table in SQL or a spreadsheet in Excel.

Example:

```
data = {"Name": ["A", "B"], "Age": [20, 25]}
df = pd.DataFrame(data)
```

4. Reading and Writing Files

Pandas can read data from various file formats such as CSV, Excel, JSON, SQL databases, etc.

Common functions:

```
pd.read_csv()
pd.read_excel()
df.to_csv()
df.to_excel()
```

5. Data Inspection

After loading data, inspection is important.

```
df.head() - shows first 5 rows
df.tail() - shows last 5 rows
df.info() - summary of dataset
df.describe() - statistical summary
df.shape - returns (rows, columns)
```

6. Data Selection and Filtering

You can select specific columns using `df['column_name']`.
Row selection can be done using:
`loc[]` - label-based selection
`iloc[]` - position-based selection
Conditional filtering:
`df[df['Age'] > 21]`

7. Handling Missing Data

Missing data is common in real-world datasets.
`df.isnull()` - detects missing values
`df.dropna()` - removes missing values
`df.fillna(value)` - replaces missing values with a specified value

8. Sorting and Grouping

Sorting:
`df.sort_values('Age')`
Grouping:
`df.groupby('Department')['Salary'].mean()`
Aggregation:
`df.groupby('Department').agg(['mean', 'sum', 'max'])`

9. Merging and Concatenation

Merging is similar to SQL JOIN.
`pd.merge(df1, df2, on='id', how='inner')`
Concatenation:
`pd.concat([df1, df2])`

10. Apply and Lambda Functions

The `apply()` function allows applying a function along an axis.
Example:
`df['Age'].apply(lambda x: x + 1)`

11. Pivot Tables

Pivot tables summarize data.
`pd.pivot_table(df, values='Salary', index='Department', aggfunc='mean')`

12. Working with Dates

Date handling is important in time-series analysis.
`df['date'] = pd.to_datetime(df['date'])`

```
df['year'] = df['date'].dt.year  
df['month'] = df['date'].dt.month
```

13. Advanced Topics

MultiIndex - hierarchical indexing
Categorical data type - memory efficient
Vectorized operations - faster computation
Performance optimization - avoid loops, use built-in functions